# Lecture 14

# Single-Source Shortest Paths

Slides are based on the textbook and its notes

## Overview

- ***Shortest paths***: How to find the shortest route between two points on a map.
- **Input** to a ***shortest-paths problem***:
  - Directed graph $G = (V, E)$
  - Weight function $w : E \to \mathbb{R}$
- ***Weight of path*** $p = \langle v_0, v_1, \ldots, v_k \rangle =$ $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$ .

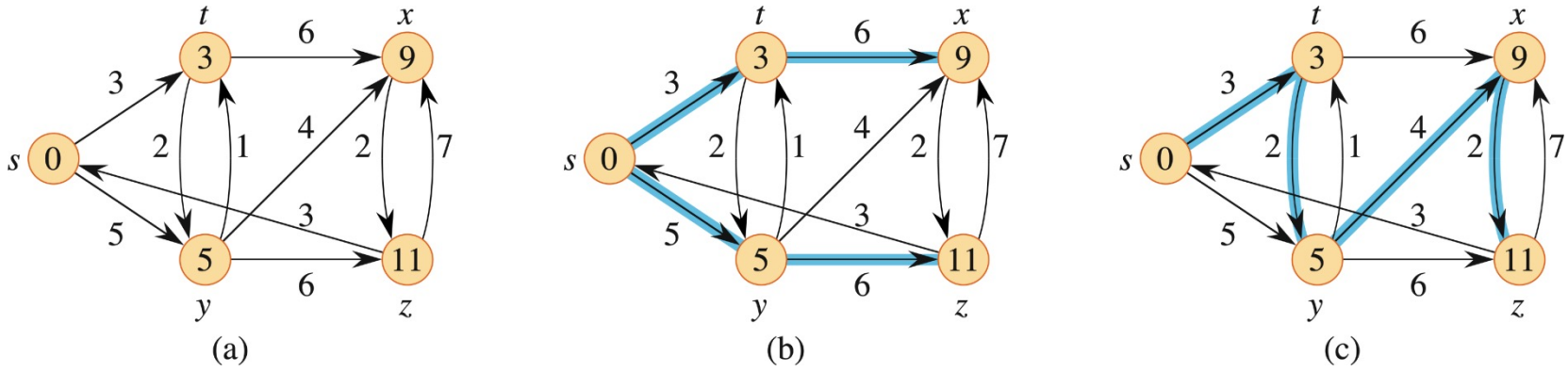
  = sum of edge weights on path $p$.
- ***Shortest-path weight*** $\delta(u, v)$ from $u$ to $v$:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{if there is a path from } u \text{ to } v \text{ ,} \\ \infty & \text{otherwise .} \end{cases}$$

A ***shortest path*** from $u$ to $v$ is any path $p$ such that $w(p) = \delta(u, v)$.

# Shortest paths: Example

- ***Example***
  - (a) A weighted, directed graph with shortest-path weights from source *s*.
  - (b) The blue edges form a shortest-paths tree rooted at the source *s*.
  - (c) Another shortest-paths tree with the same root.
  - Can think of weights as representing any measure that
    - accumulates linearly along a path, and
    - we want to minimize.
    - Examples: time, cost, penalties, loss.
    - Generalization of breath-first search to weighted graphs.



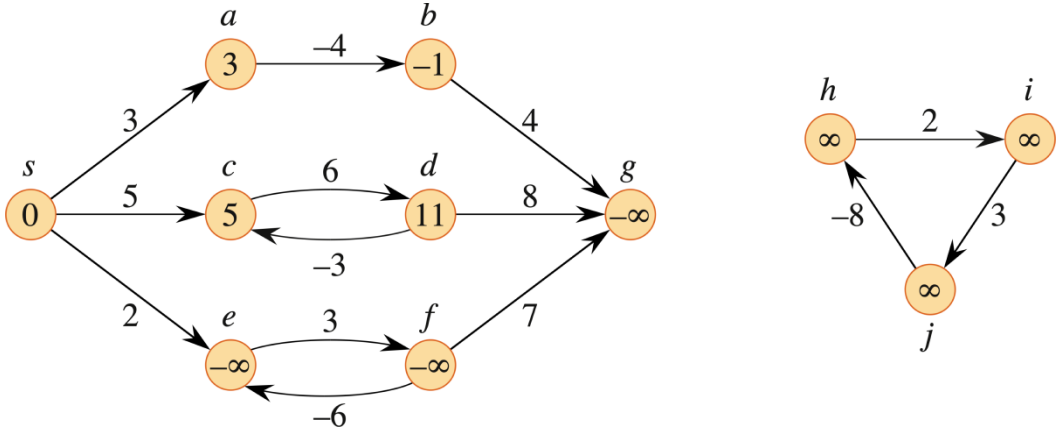
## Shortest paths: Variants

- **Variants**
  - ***Single-source***: Find shortest paths from a given source vertex $s \in V$ to every vertex $v \in V$.
  - ***Single-destination***: Find shortest paths to a given destination vertex.
  - ***Single-pair***: Find shortest path from $u$ to $v$. No way known that's better in worst case than solving single-source.
  - ***All-pairs***: Find shortest path from $u$ to $v$ for all $u, v \in V$.

- **Optimal substructure**
  - ***Lemma***: Any subpath of a shortest path is a shortest part.

# Shortest paths: Negative-weight edges

- **Negative-weight edges**: Edges whose weights are negative.
  - The shortest-path weight $\delta(u, v)$ remains well defined as long as no negative-weight cycles are reachable from the source $s$.
  - If we have a negative-weight cycle, we can just keep going around it, and get $w(s, v) = -\infty$ for all $v$ on the cycle.




**Negative edge weights in a directed graph**. The shortest-path weight from source $s$ appears within each vertex. Because vertices $e$ and $f$ form a negative-weight cycle reachable from $s$, they have shortest-path weight of $-\infty$. Because vertex $g$ is reachable from a vertex whose shortest-path weight is $-\infty$, it, too, has a shortest-path weight of $-\infty$. Vertices such as $h$, $i$, and $j$ are not reachable from $s$, and so their shortest-path weights are $\infty$, even though they lie on a negative-weight cycle.

# Output of single-source shortest-path algorithm

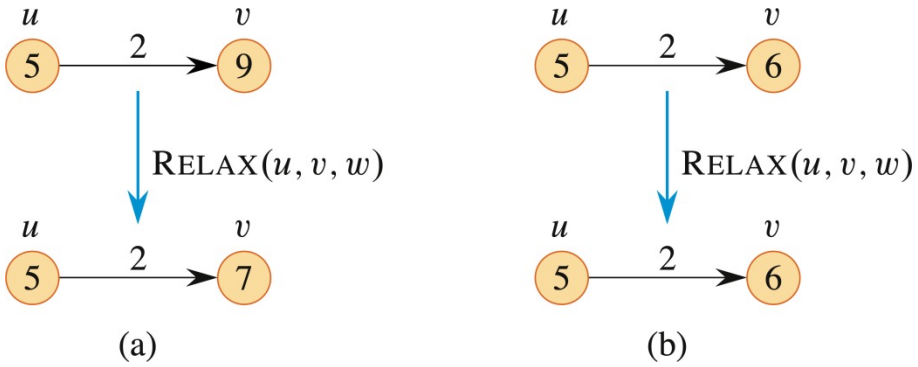
- For each vertex $v \in V$:
  - $v.d = \delta(s, v)$.
    - Initially, $v.d = \infty$.
    - Reduces as algorithms progress. But always maintain $v.d \geq \delta(s, v)$.
    - Call $v.d$ a ***shortest-path estimate*** (upper-bound on the weight of a shortest path from source $s$ to $v$.)
  - $v.\pi$ = predecessor of $v$ on a shortest path from $s$.
    - If no predecessor, $v.\pi$ = NIL.
    - $\pi$ induces a tree – ***shortest-path tree***.
- **Initialization**: All the shortest-paths algorithms start with:

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

# Shortest-paths: Relaxation

- The process of ***relaxing*** an edge $(u, v)$ consists of testing whether going through vertex $u$ improves the shortest path to vertex $v$ found so far and, if so, updating $v.d$ and $v.\pi$.

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2        v.d = u.d + w(u, v)
3        v.π = u
```




- ***Example***: Relaxing an edge $(u, v)$ with weight $w(u, v) = 2$. The shortest-path estimate of each vertex appears within the vertex.
  - (a) Because $v.d > u.d + w(u, v)$ prior to relaxation, the value of $v.d$ decreases.
  - (b) Since we have $v.d \leq u.d + w(u, v)$ before relaxing the edge, the relaxation step leaves $v.d$ unchanged.

# Shortest-paths: Properties

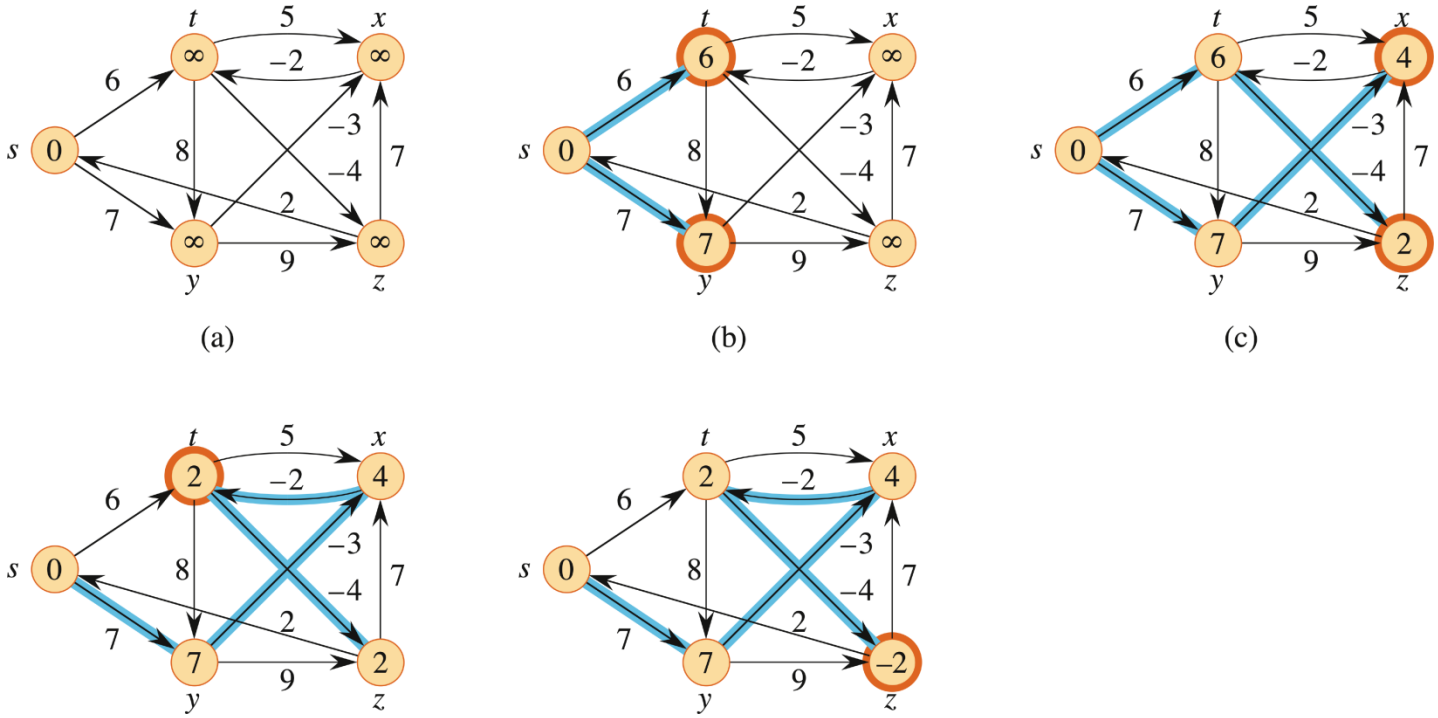
- **Triangle inequality**: For all $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-bound property**: Always have $v.d \geq \delta(s, v)$ for all $v$. Once $v.d$ gets down to $\delta(s, v)$, it never changes.
- **No-path property**: If $\delta(s, v) = \infty$, then $v.d = \infty$ always.
- **Convergence property**: If $s \rightsquigarrow u \rightarrow v$ is a shortest path, $u.d = \delta(s, u)$, and edge $(u, v)$ is relaxed, then $v.d = \delta(s, v)$ afterward.
- **Path-relaxation property**: Let $p = \langle v_0, v_1, \ldots, v_k \rangle$ be a shortest path from $s = v_0$ to $v_k$. If the edges of $p$ are relaxed, *in the order*, $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $v_k.d = \delta(s, v_k)$.

# The Bellman-Ford algorithm

- The ***Bellman-Ford algorithm*** solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
  - It computes $v.d$ and $v.\pi$ for all $v \in V$.
  - It returns TRUE if no negative-weight cycles reachable from $s$, FALSE otherwise.
  - Time: $O(V^2 + VE)$. The first for loop makes $|V| - 1$ passes over the edges, and each pass takes $\Theta(V + E)$ time. We use $O$ rather than $\Theta$ because sometimes $< |V| - 1$ passes are enough.

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

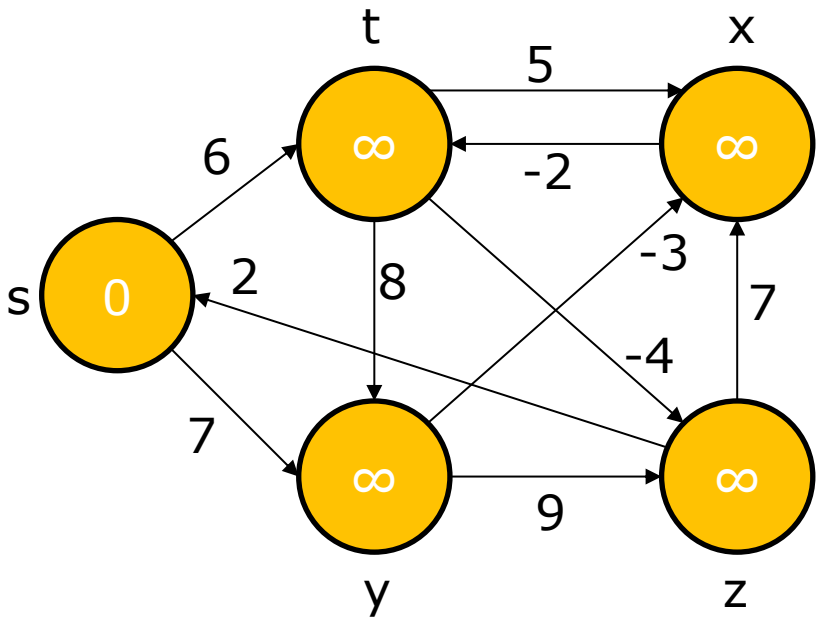
# The Bellman-Ford algorithm: Example




**The execution of the Bellman-Ford algorithm**. The source is vertex $s$. The $d$ values appear within the vertices, and blue edges indicate predecessor values: if edge $(u, v)$ is blue, then $v.\pi = u$. In this particular example, each pass relaxes the edges in the order $(t, x)$, $(t, y)$, $(t, z)$, $(x, t)$, $(y, x)$, $(y, z)$, $(z, x)$, $(z, s)$, $(s, t)$, $(s, y)$. **(a)** The situation just before the first pass over the edges. **(b)-(e)** The situation after each successive pass over the edges. Vertices whose shortest-path estimates and predecessors have changed due to a pass are highlighted in orange. The $d$ and $\pi$ values in part (e) are the final values. The Bellman-Ford algorithm returns TRUE in this example.

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

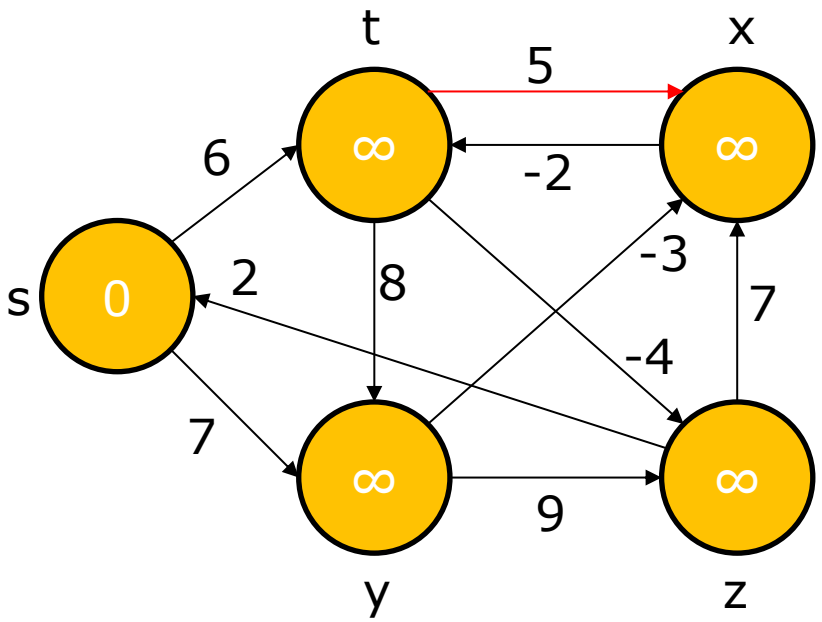



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

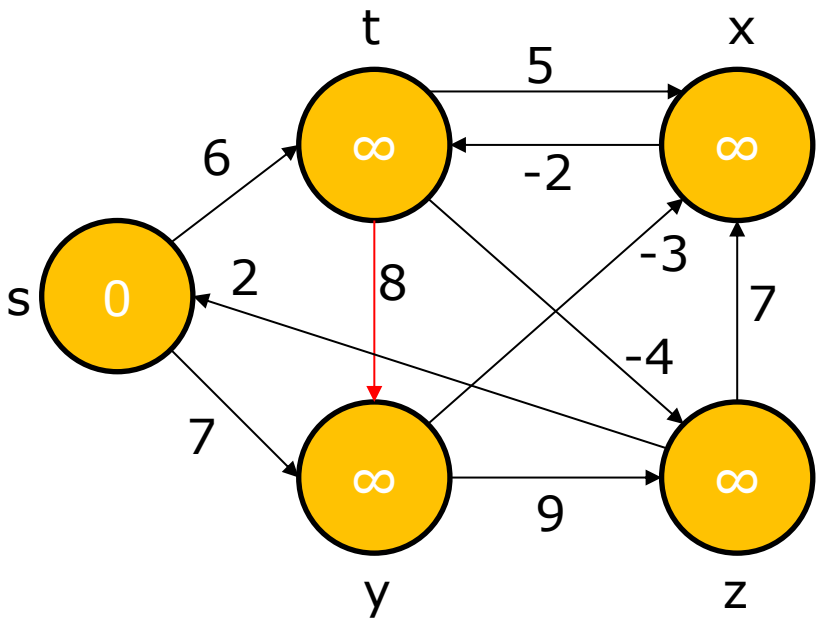



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

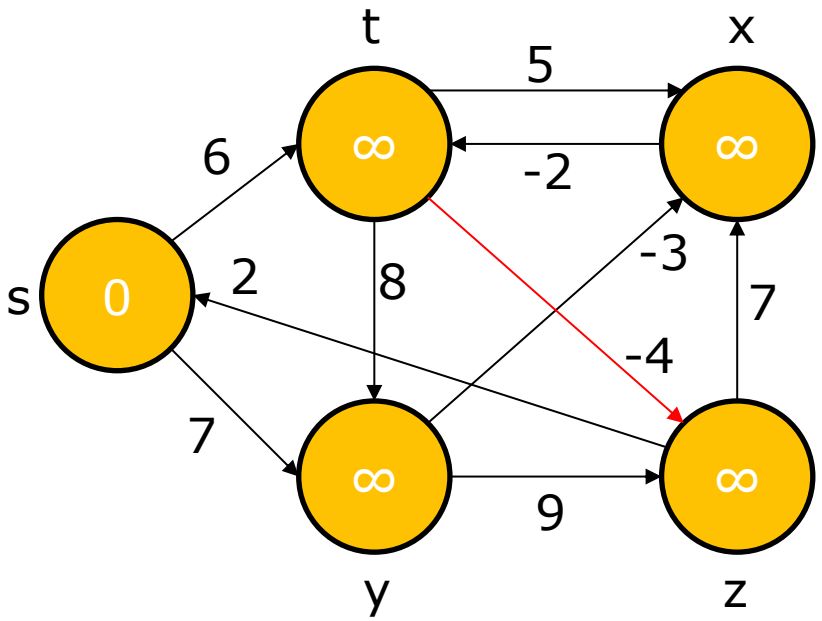



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

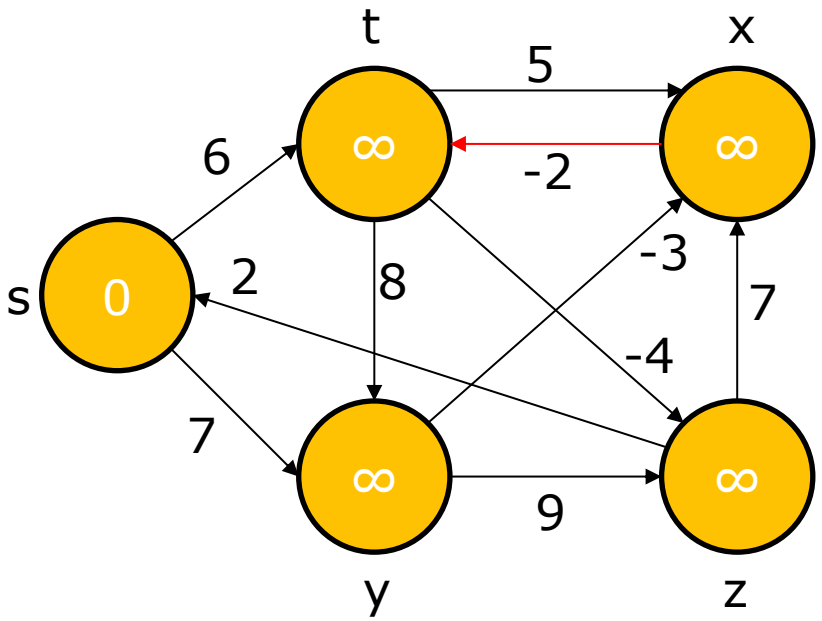



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

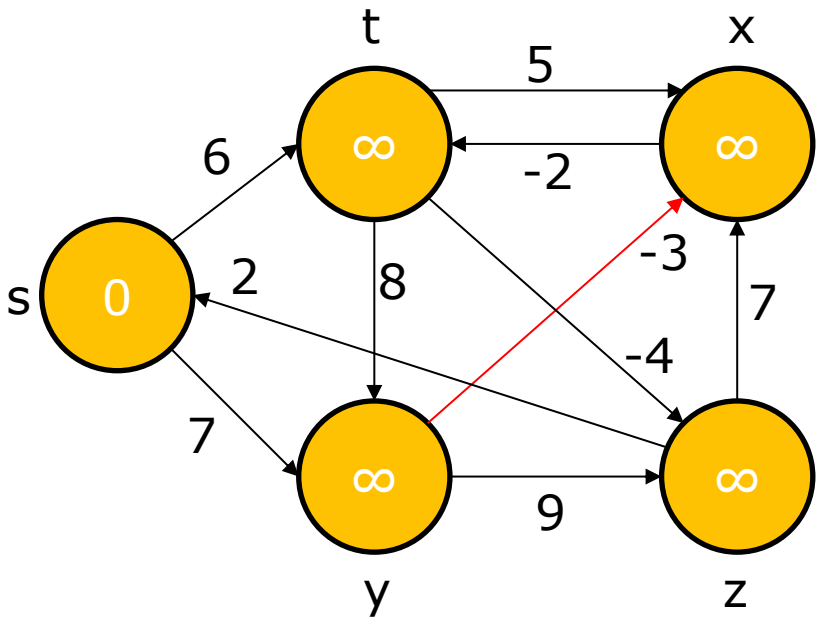



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

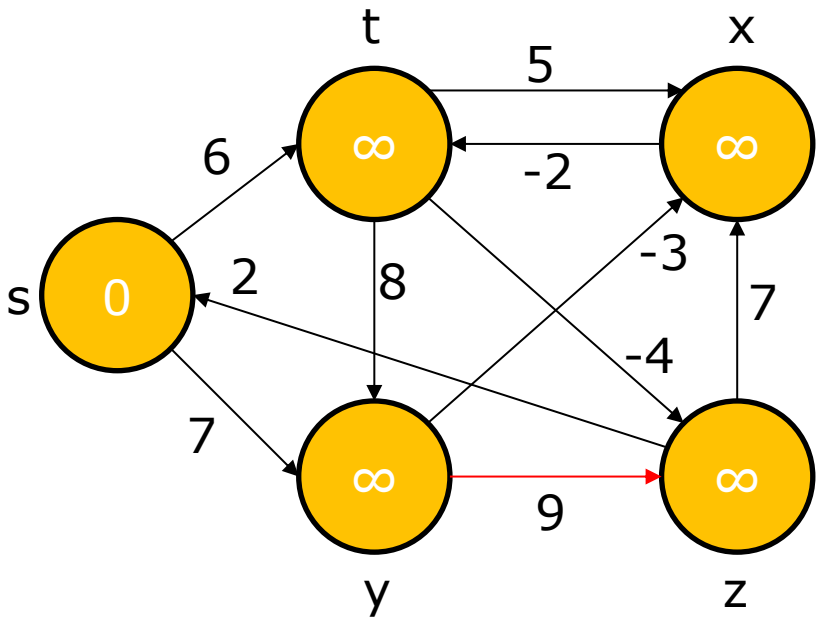



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

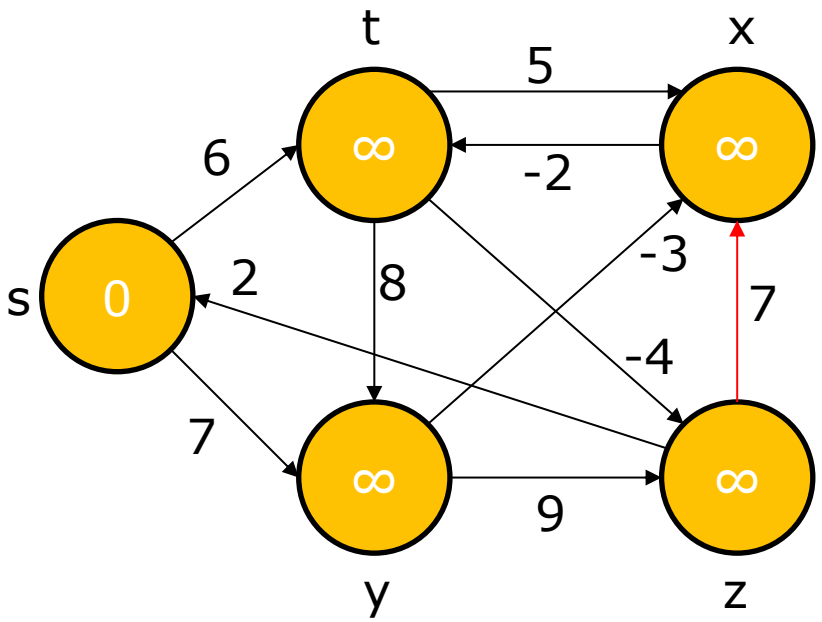



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

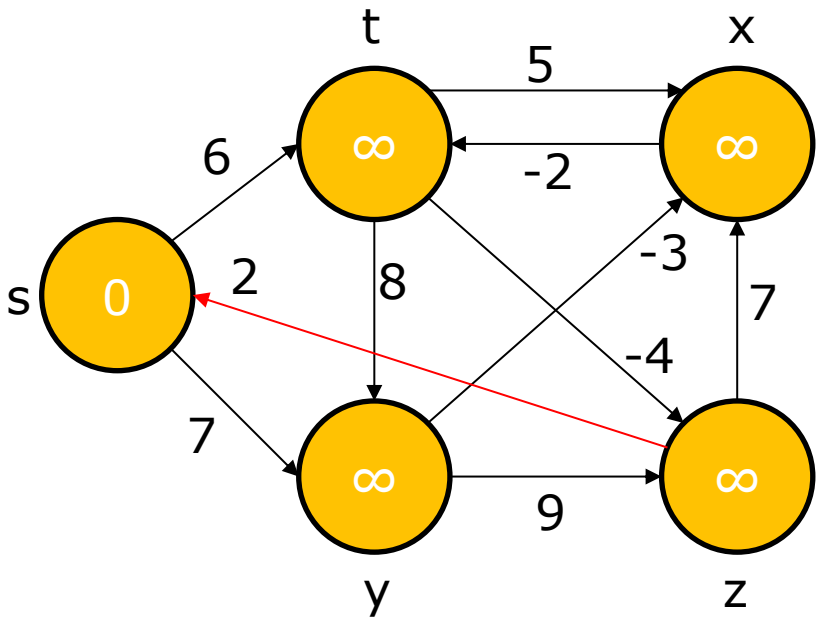



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

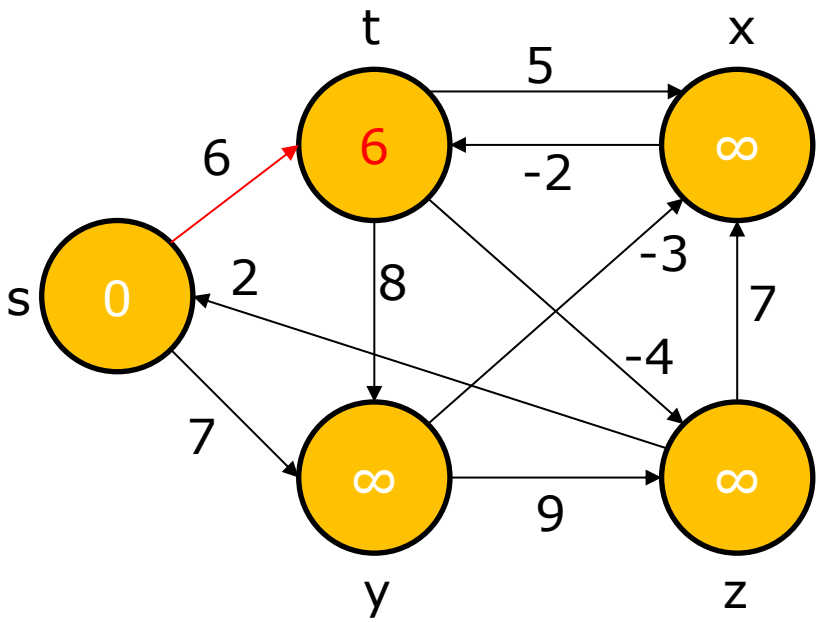



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | ∞ | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | Nil | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

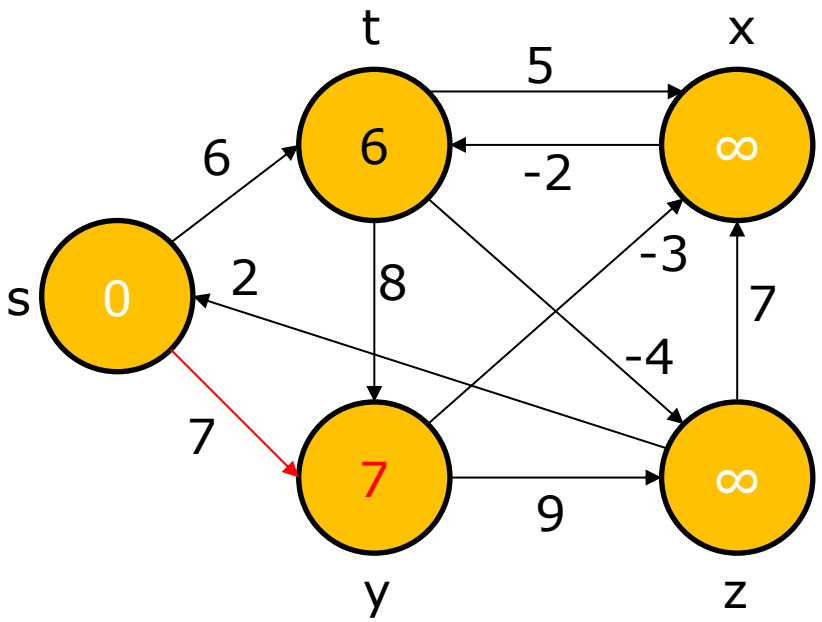



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | ∞ | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | Nil | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

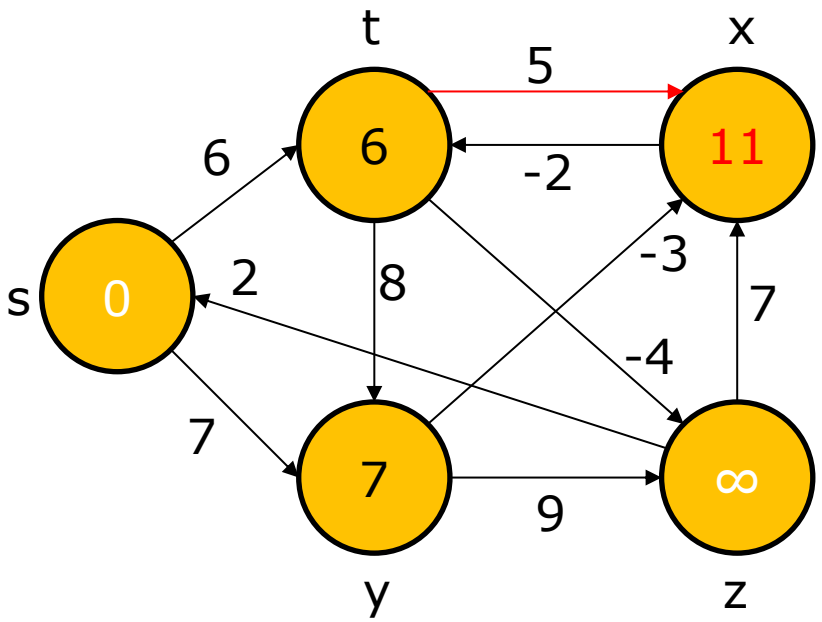



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

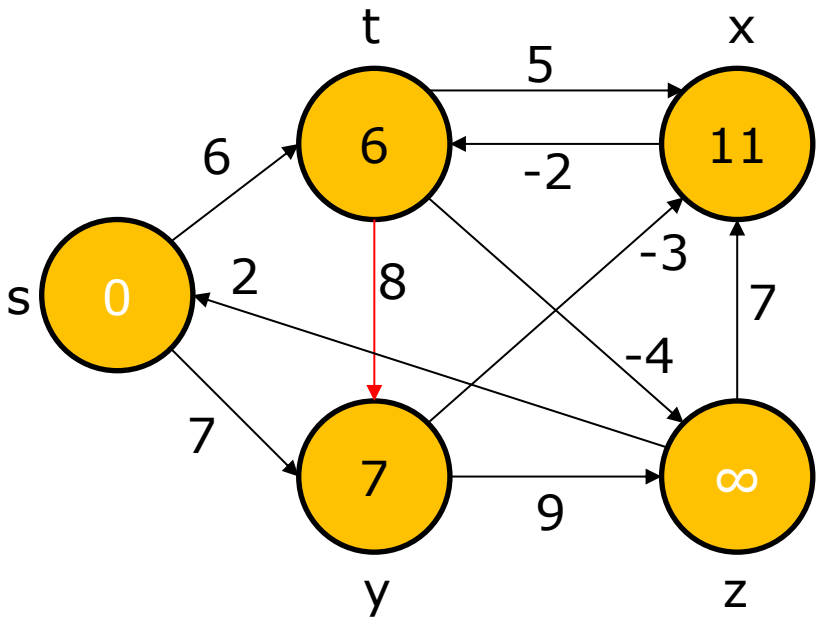



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 11 | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | t | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

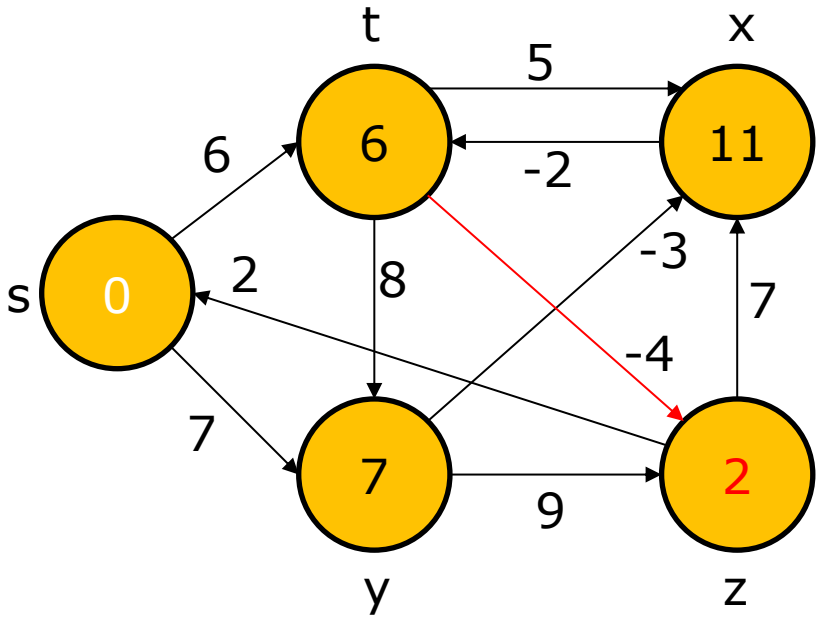



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 11 | | |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | t | | |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

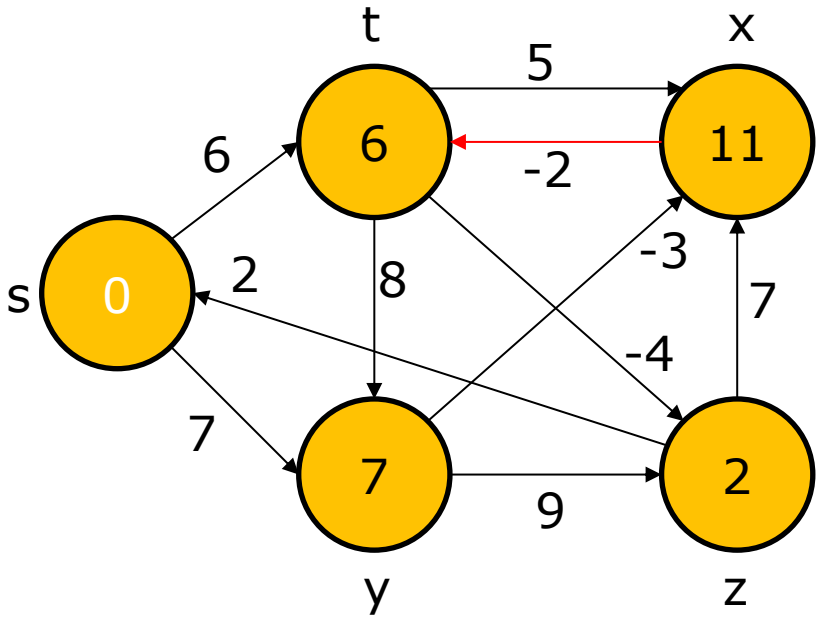



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 11 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | t | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

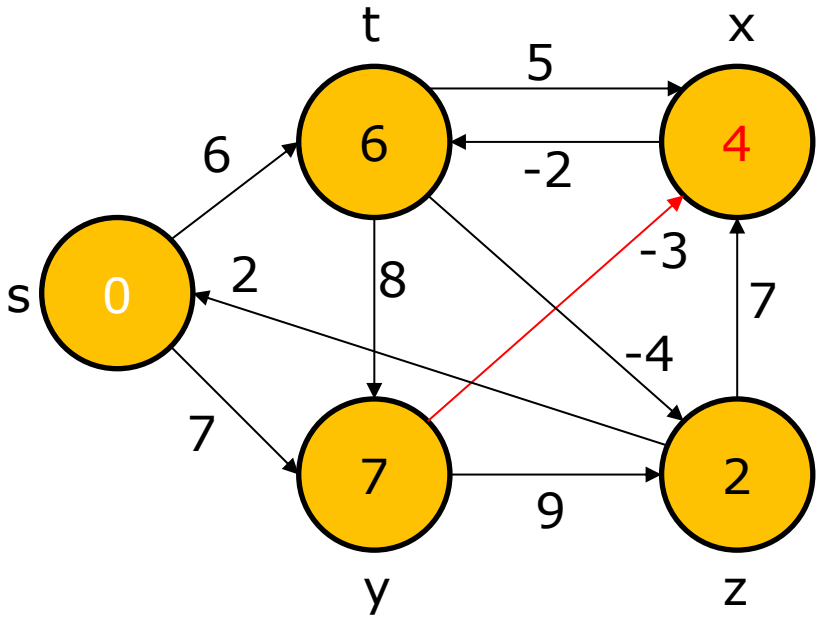



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 11 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | t | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

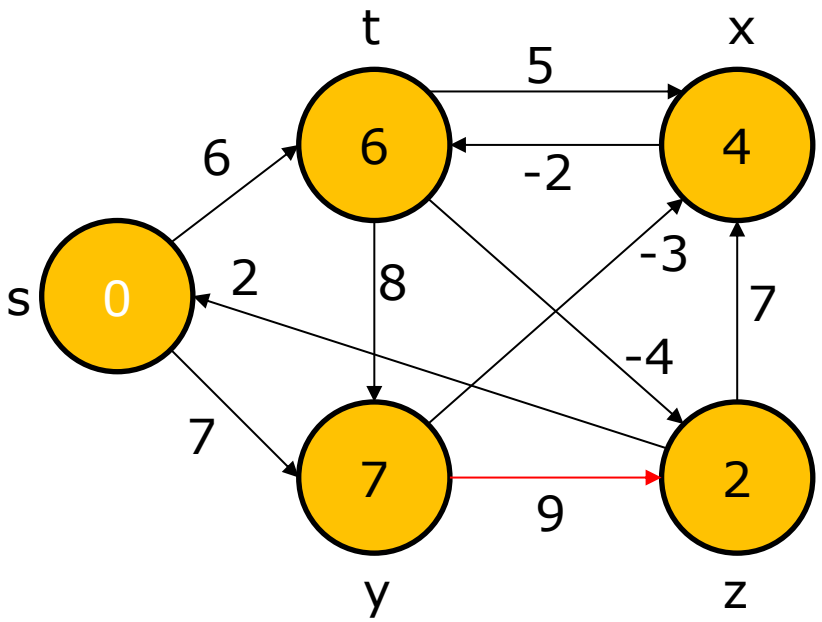



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 4 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | y | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

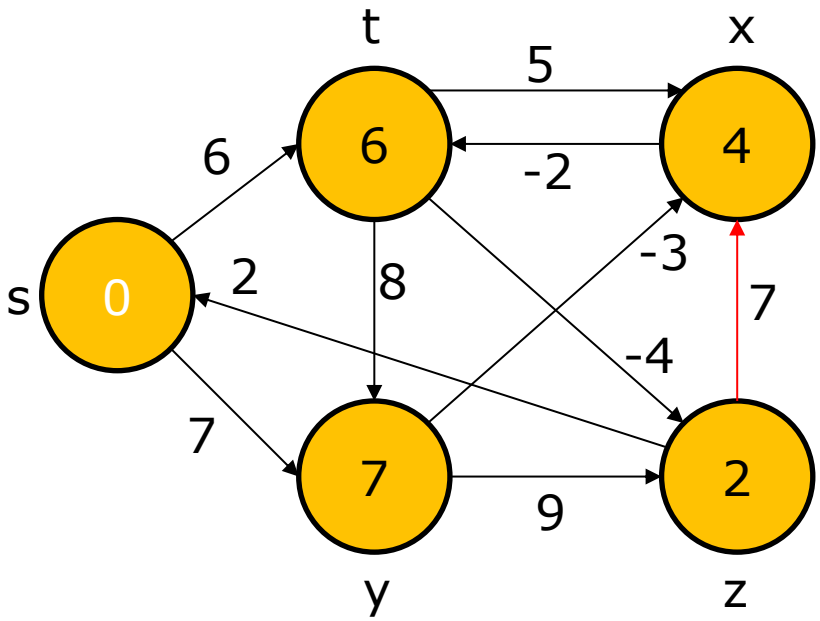



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 4 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | y | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

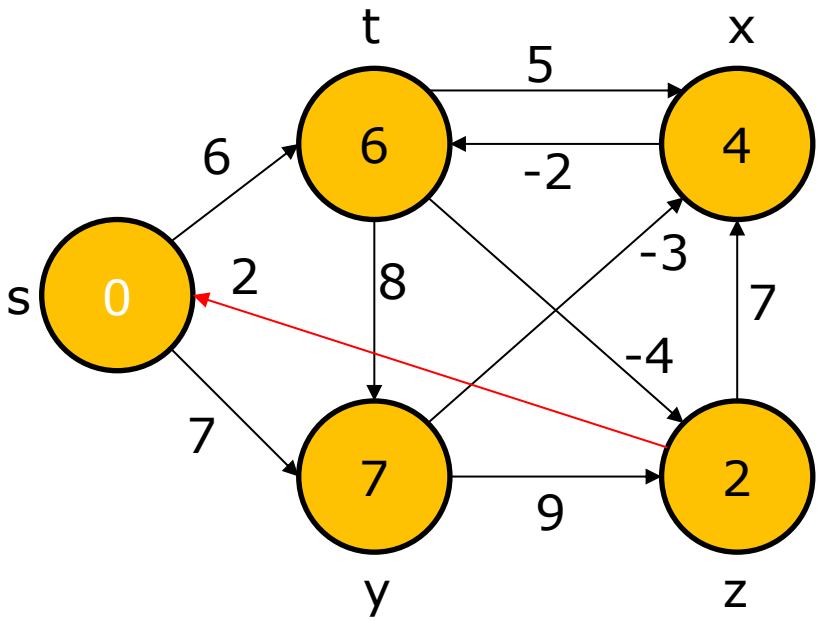



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 4 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | y | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

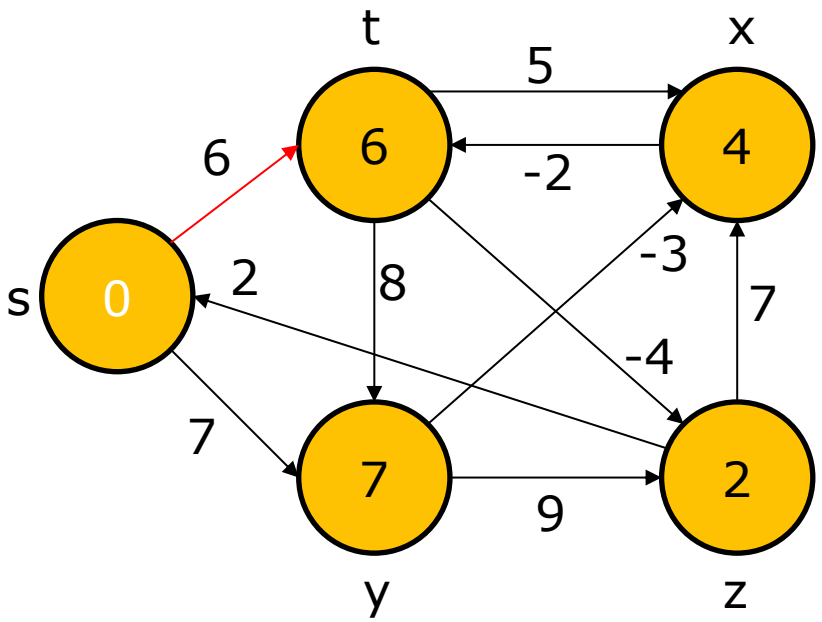



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 4 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | y | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

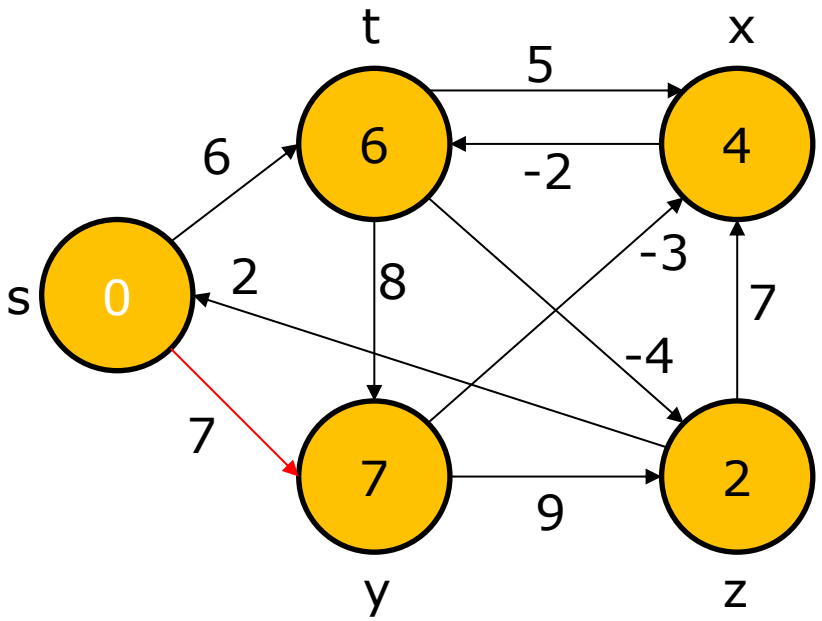



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | | | 4 | | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | y | | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

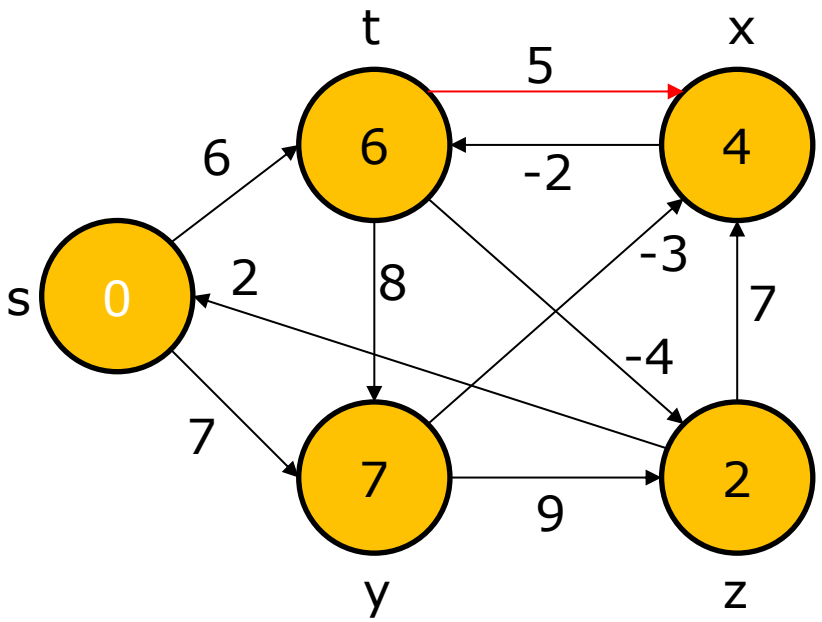



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

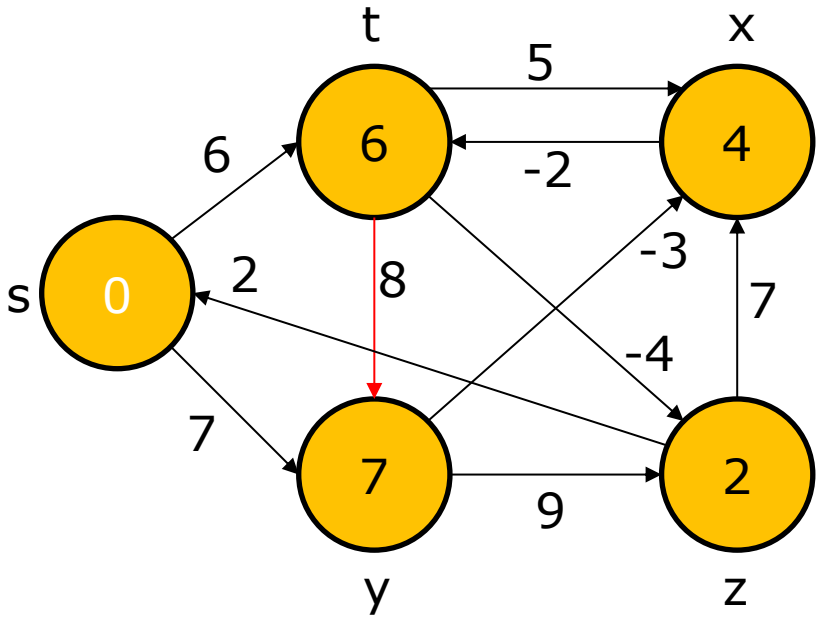



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

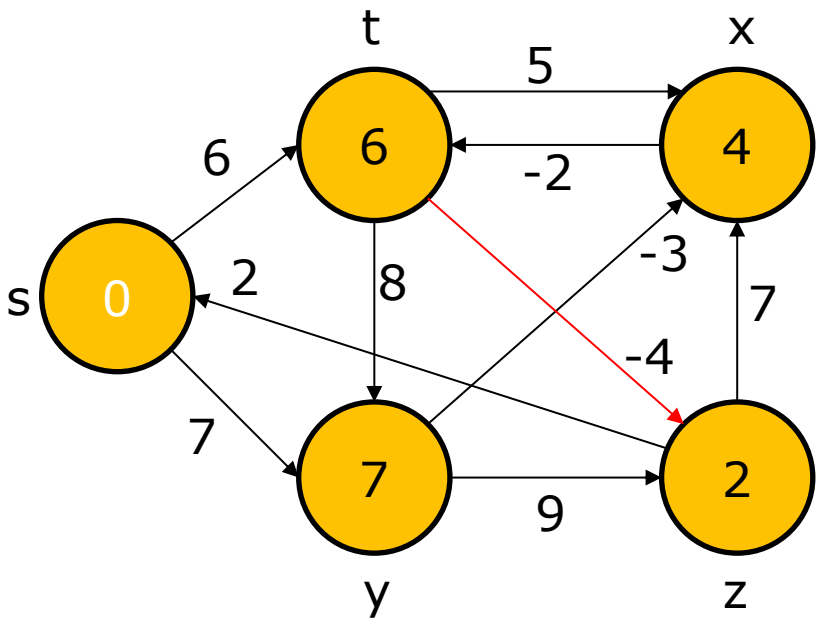



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

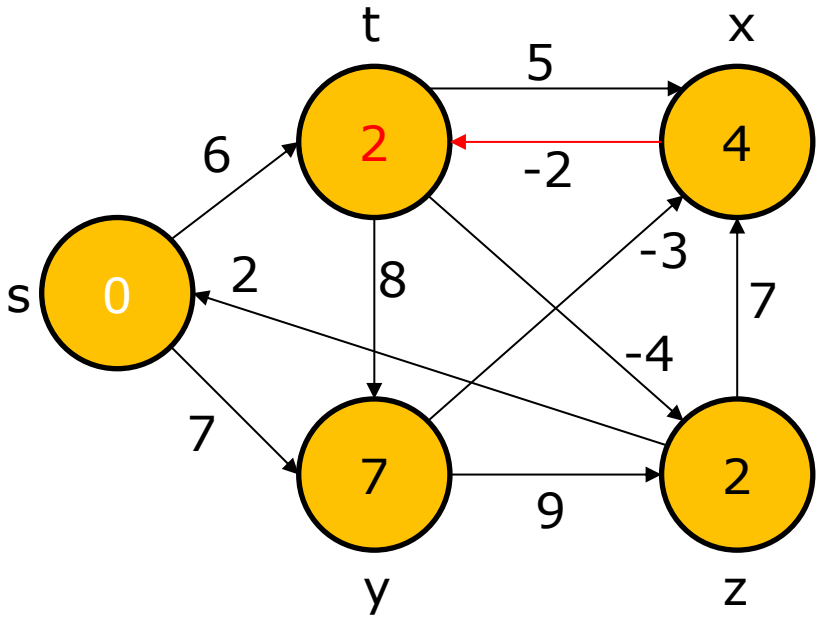



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

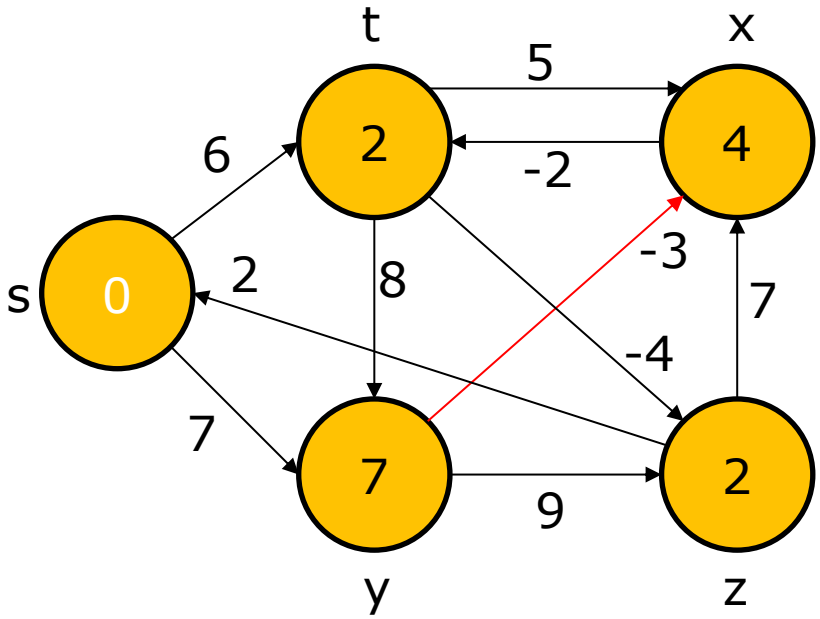



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

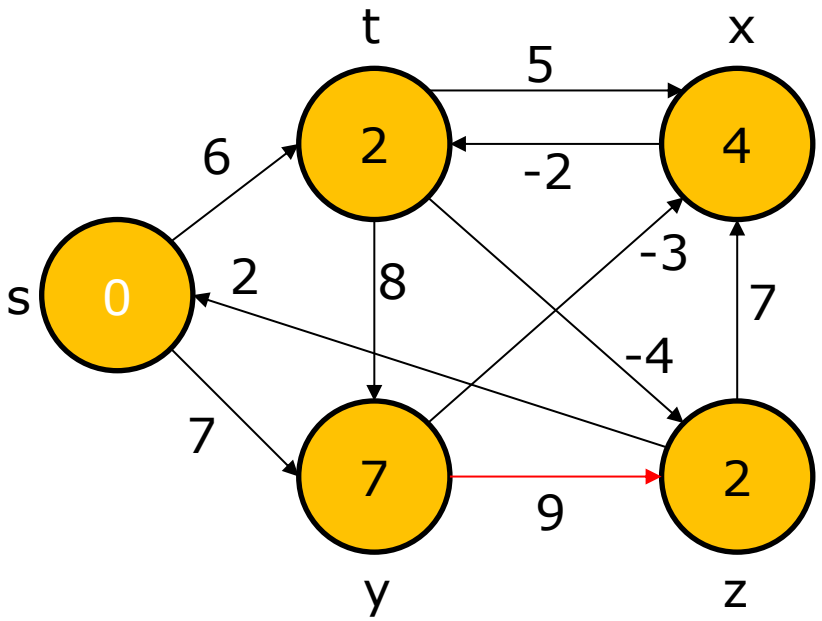



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

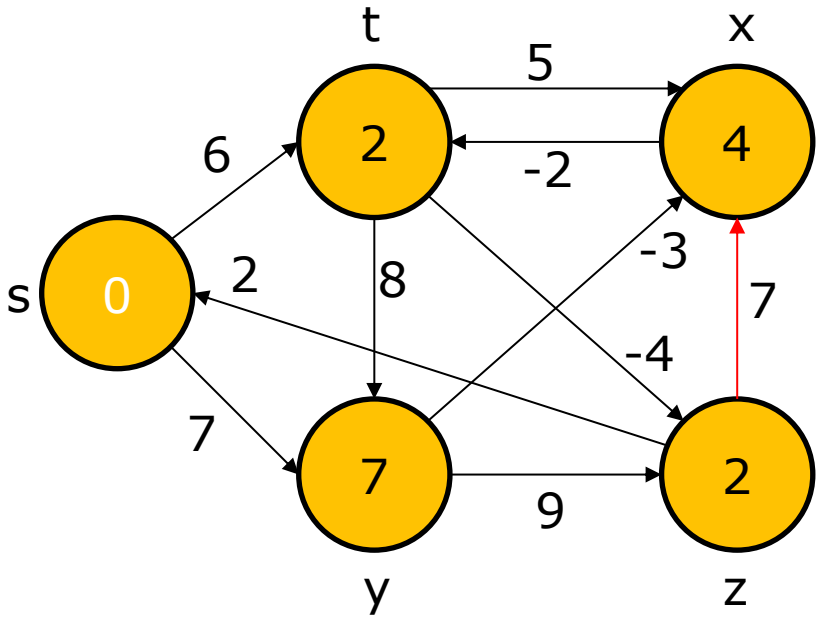



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

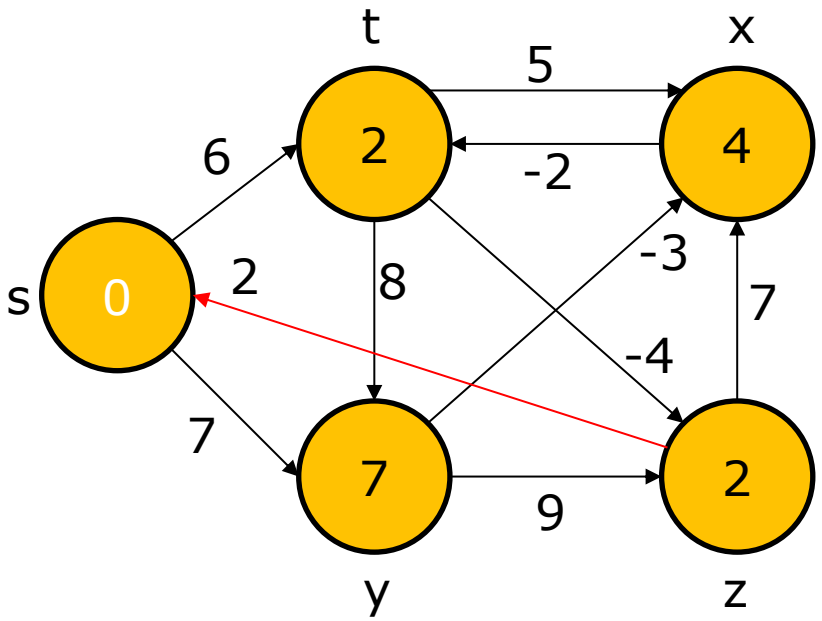



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

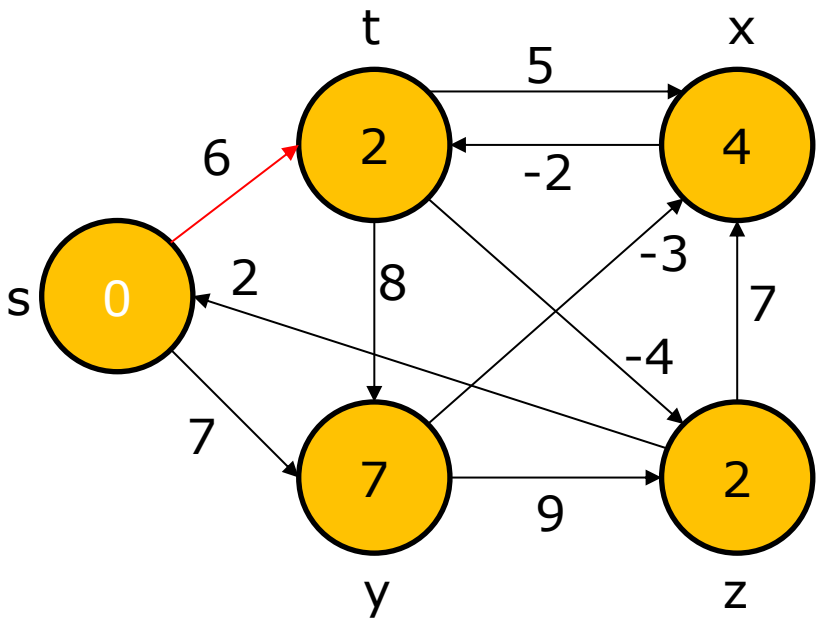



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

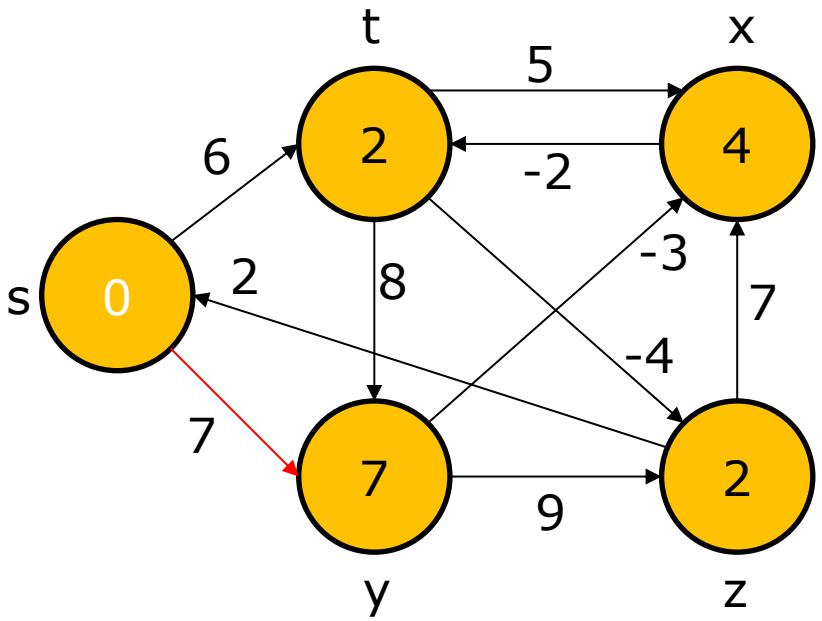



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | | 2 | | | |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | | x | | | |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

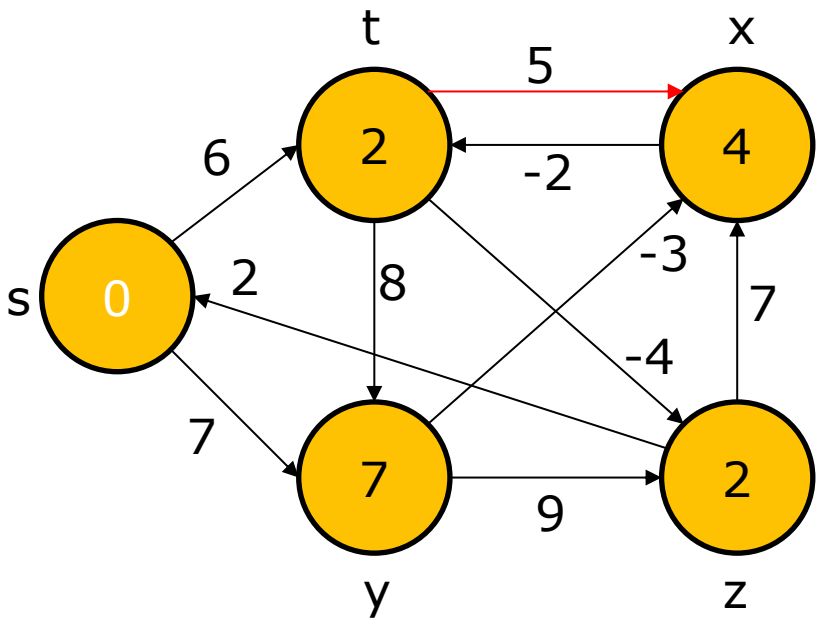



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

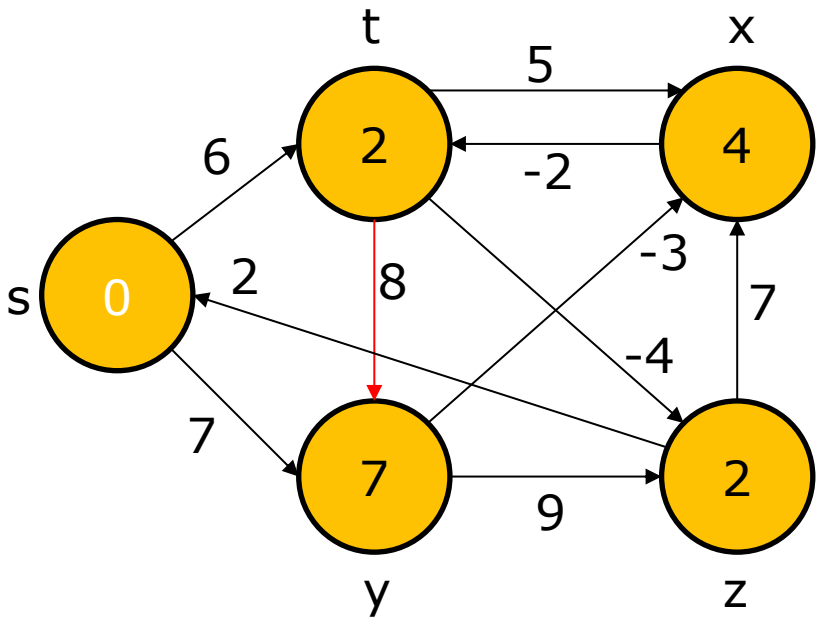



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

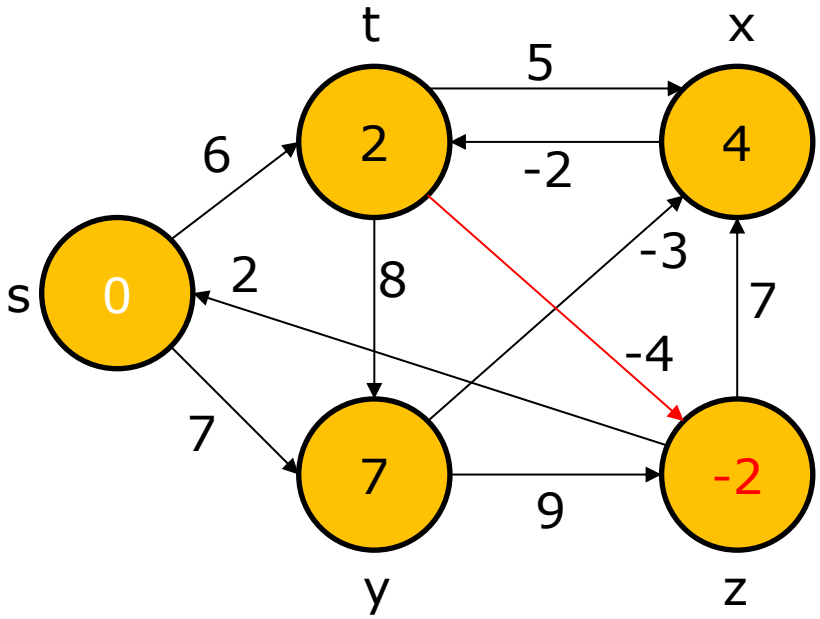



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

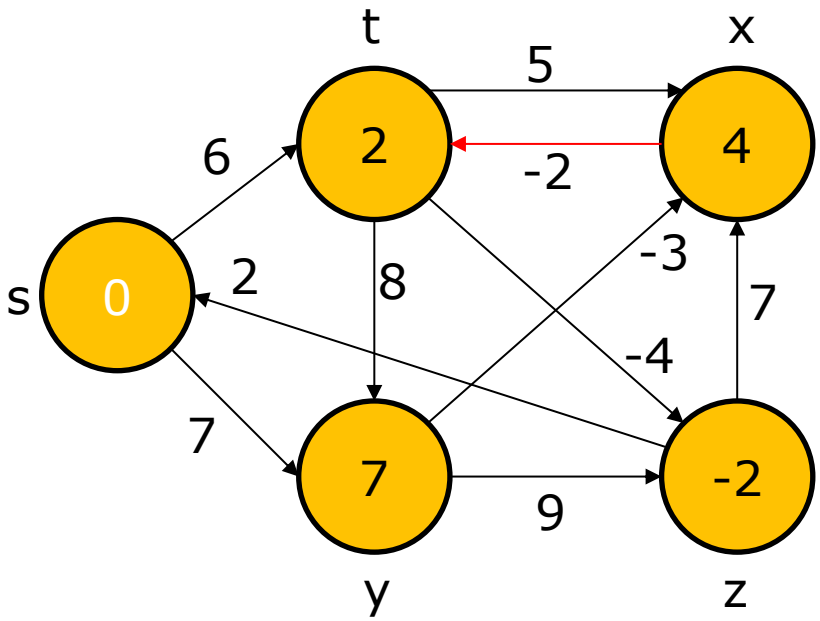



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

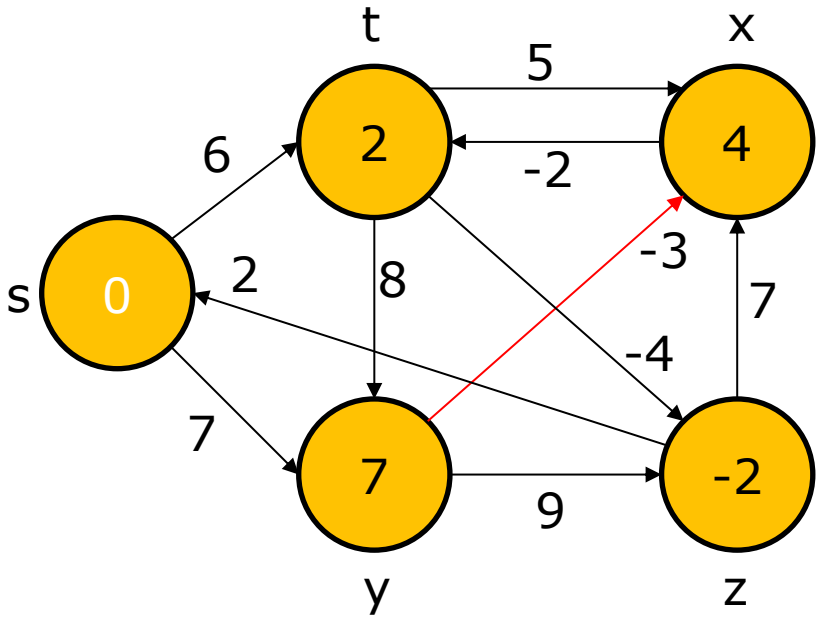



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

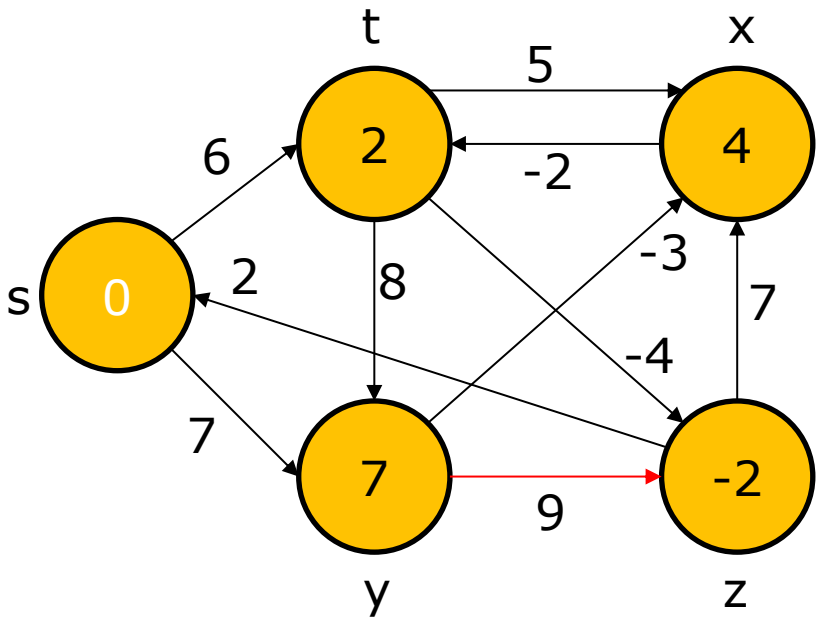



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

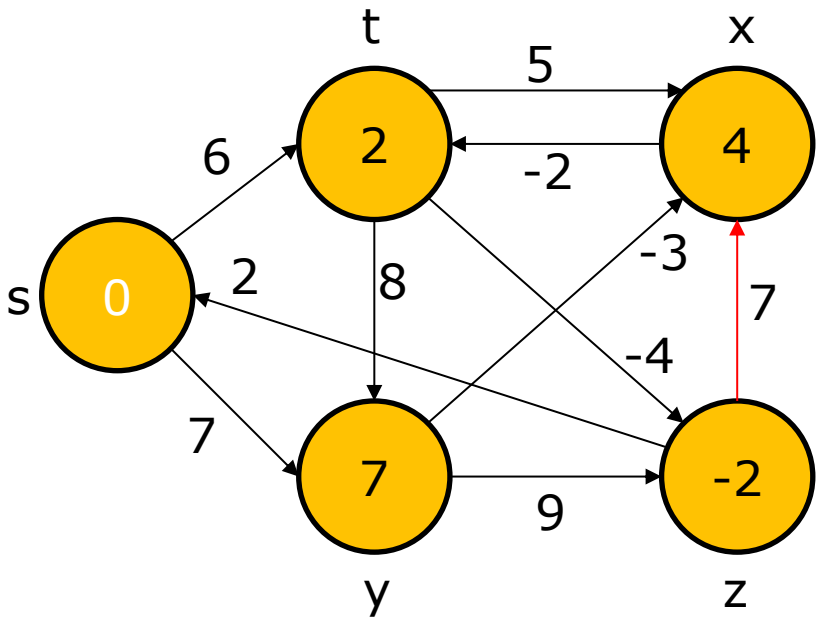



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

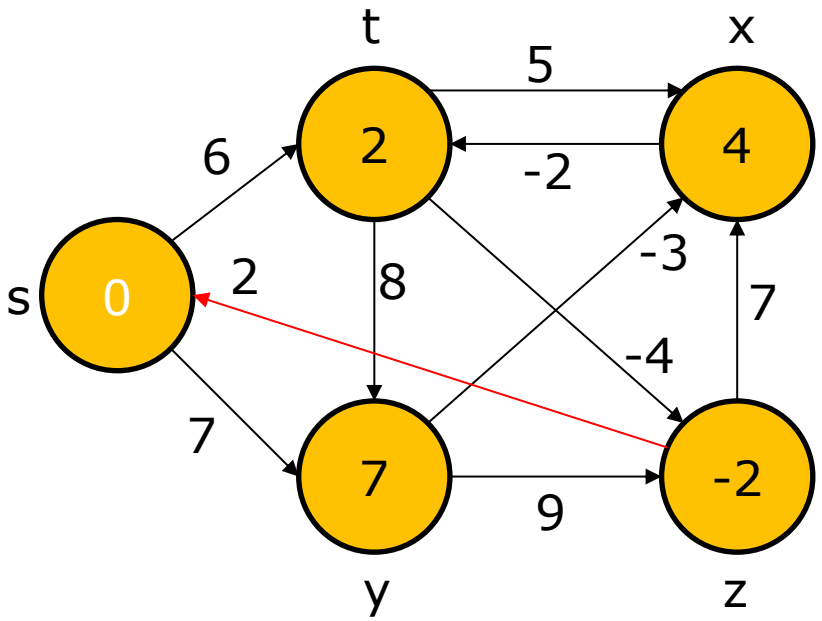



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

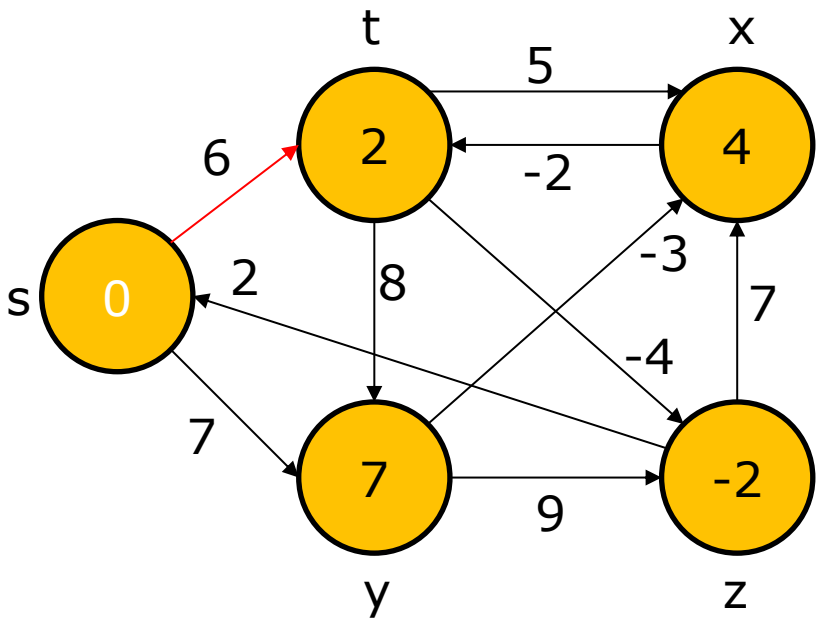



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

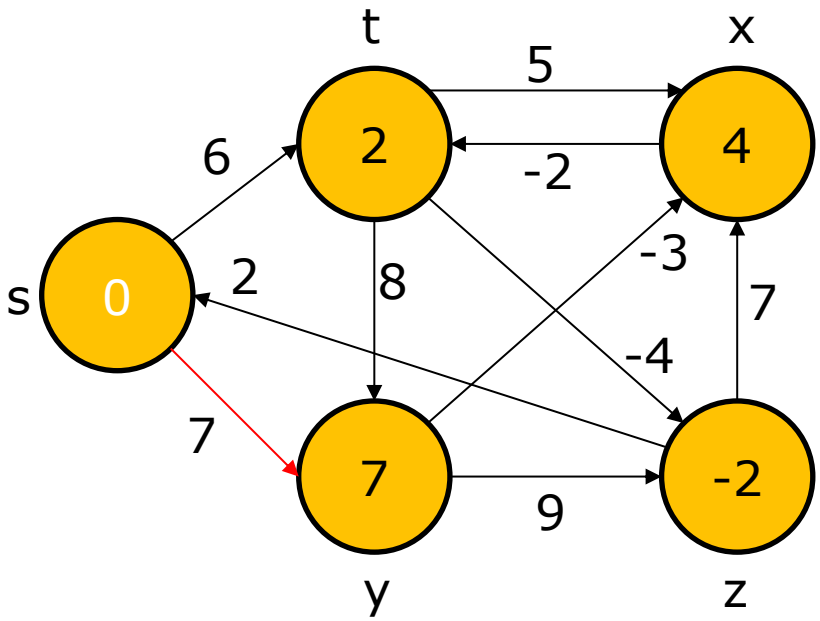



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | | | | | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | | | | | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)

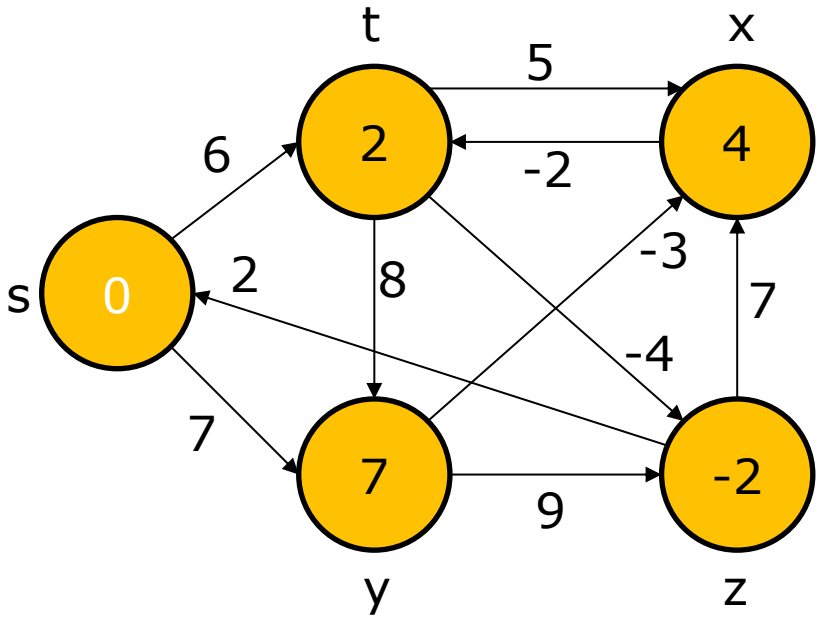



| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 6 | ∞ | 7 | ∞ |
| 2 | 0 | 6 | 4 | 7 | 2 |
| 3 | 0 | 2 | 4 | 7 | 2 |
| 4 | 0 | 2 | 4 | 7 | -2 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | s | y | s | t |
| 3 | Nil | x | y | s | t |
| 4 | Nil | x | y | s | t |

# The Bellman-Ford algorithm: Example

(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y)




```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

We need to do one last pass over the edges:

- If there is any change on any *v.d*, the algorithm returns **FALSE**.
- Otherwise it returns **TRUE**.
- In the previous example, the algorithm returns **TRUE**.

# Dijkstra's algorithm

- ***Dijkstra's algorithm*** solves the single-source shortest-paths problem on a weighted, directed graph, but it requires nonnegative weights on all edges.
  - Essentially a weighted version of breadth-first search.
  - Instead of a FIFO queue, uses a priority queue.
  - Keys are shortest-path weights ($v$, $d$).
  - Can think of waves, like BFS.
  - A wave emanates from the source.
  - The first time that a wave arrives at a vertex, a new wave emanates from the vertex.
  - The time it takes for the wave to arrive at a neighboring vertex equals the weight of the edge. (In BFS, each wave takes unit time to arrive at each neighbor.)

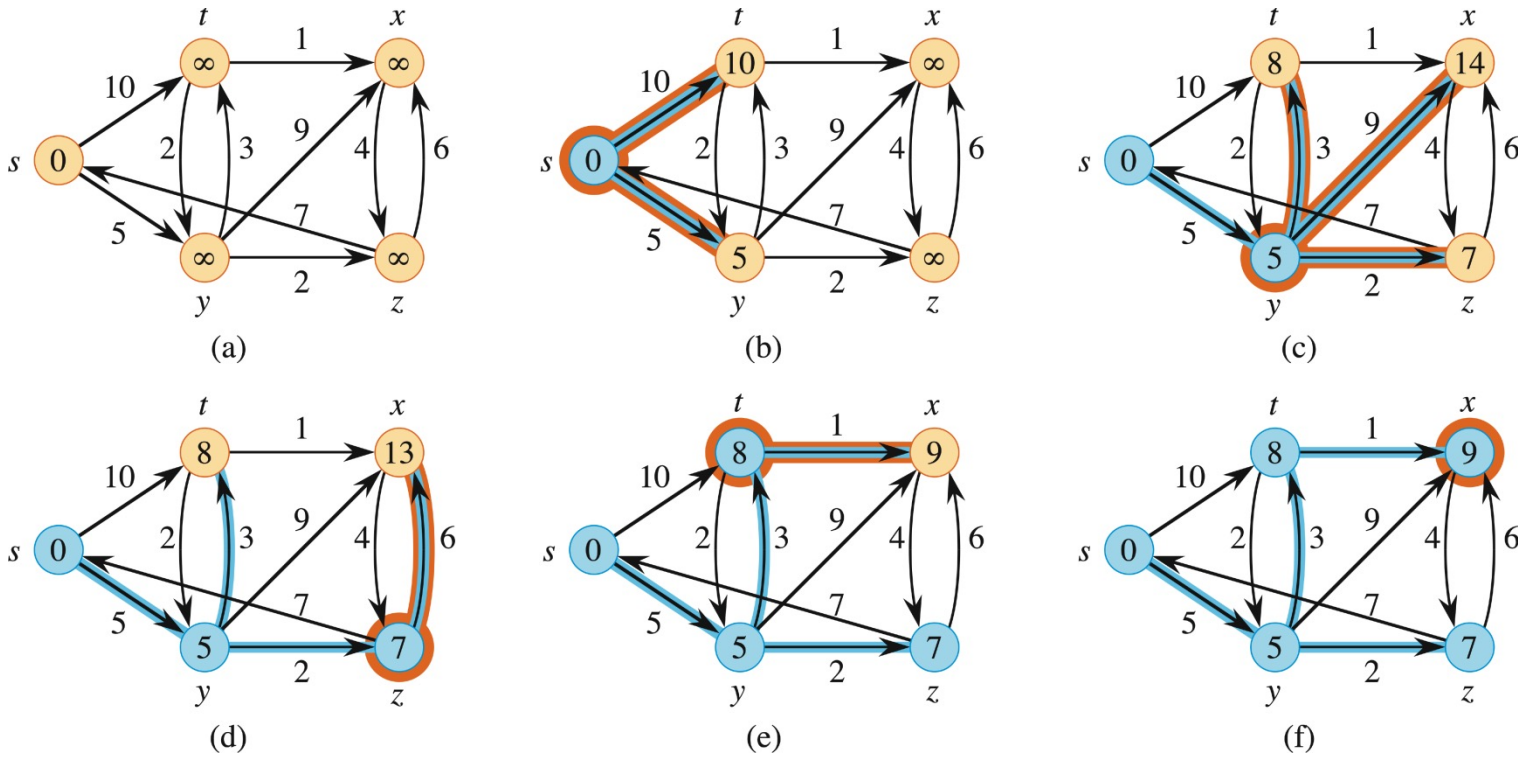
# Dijkstra's algorithm

- Have two sets of vertices:
  - $S$ = vertices whose final shortest-path weights are determined,
  - $Q$ = priority queue = $V - S$.

```
DIJKSTRA(G, w, s)
 1  INITIALIZE-SINGLE-SOURCE(G, s)
 2  S = ∅
 3  Q = ∅
 4  for each vertex u ∈ G.V
 5      INSERT(Q, u)
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      S = S ∪ {u}
 9      for each vertex v in G.Adj[u]
10          RELAX(u, v, w)
11          if the call of RELAX decreased v.d
12              DECREASE-KEY(Q, v, v.d)
```

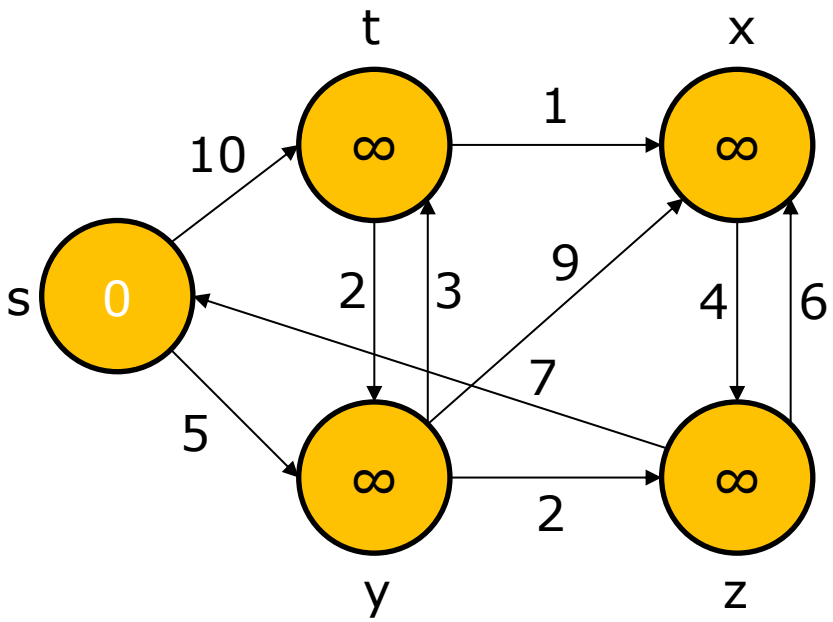
  - Looks a lot like Prim's algorithm, but computing $v.d$, and using shortest-path weights as keys.
  - Dijkstra's algorithm can be viewed as greedy, since it always chooses the "lightest" or "closest" vertex in $V - S$ to add to $S$.

# Dijkstra's algorithm: Example




**The execution of Dijkstra's algorithm**. The source $s$ is the leftmost vertex. The shortest-path estimates appear within the vertices, and blue edges indicate predecessor values. Blue vertices belong to the set $S$, and tan vertices are in the min-priority queue $Q = V - S$. **(a)** The situation just before the first iteration of the **while** loop of lines 6-12. **(b)-(f)** The situation after each successive iteration of the **while** loop. In each part, the vertex highlighted in orange was chosen as vertex $u$ in line 7, and each edge highlighted in orange caused a $d$ value and a predecessor to change when the edge was relaxed. The $d$ values and predecessors shown in part (f) are the final values.

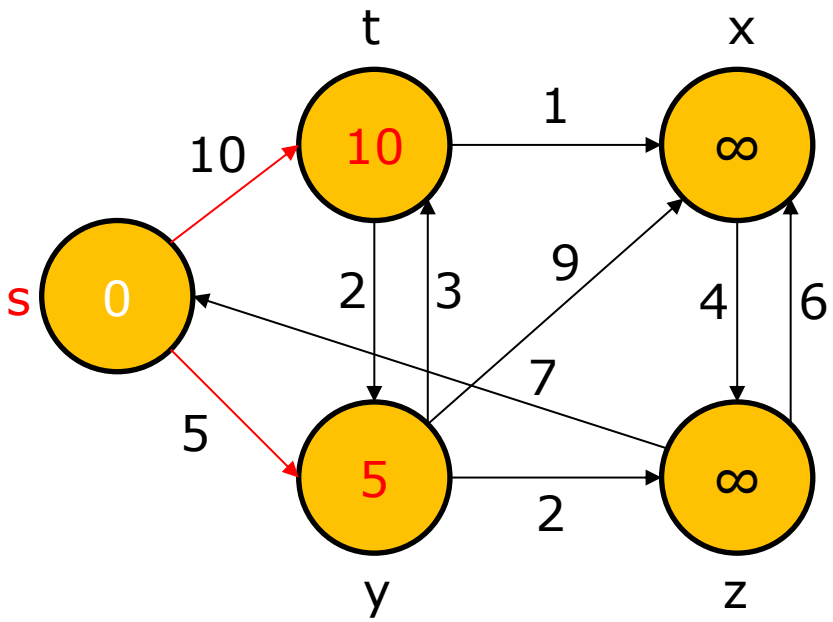
# Dijkstra's algorithm: Example




| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

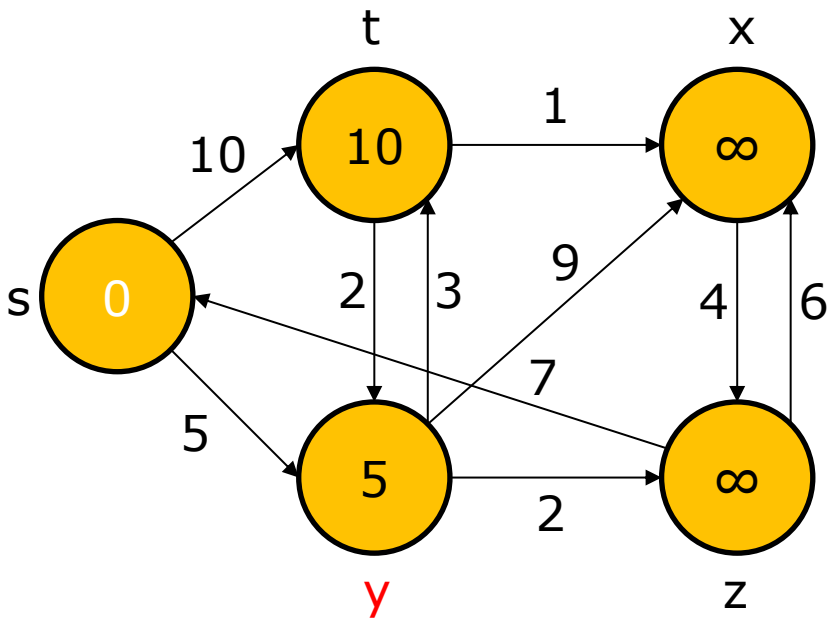



S = {s, }

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

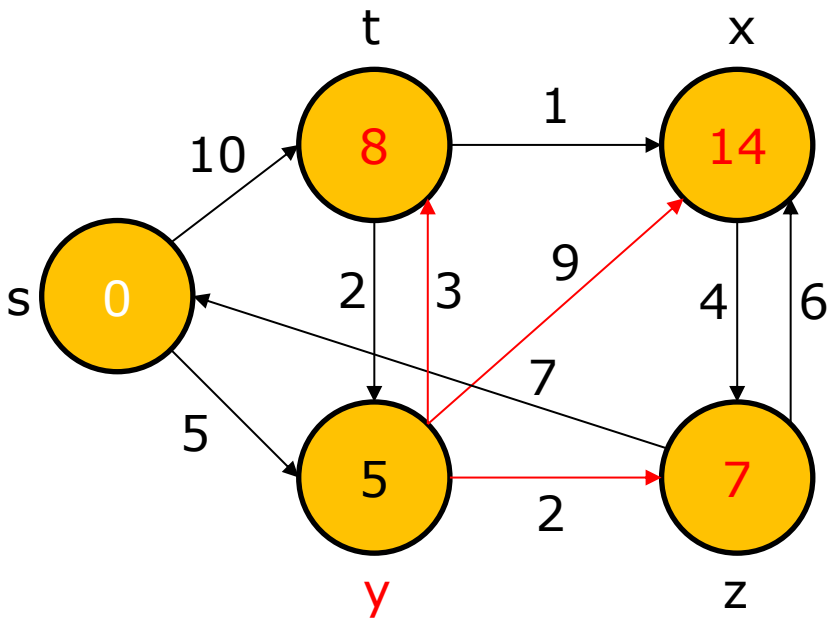



S = {s, y}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

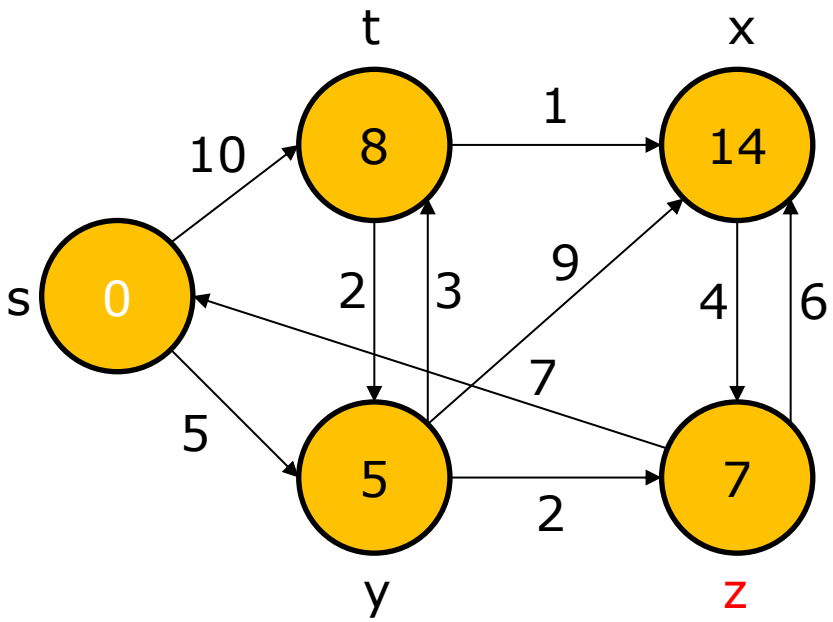



S = {s, y}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

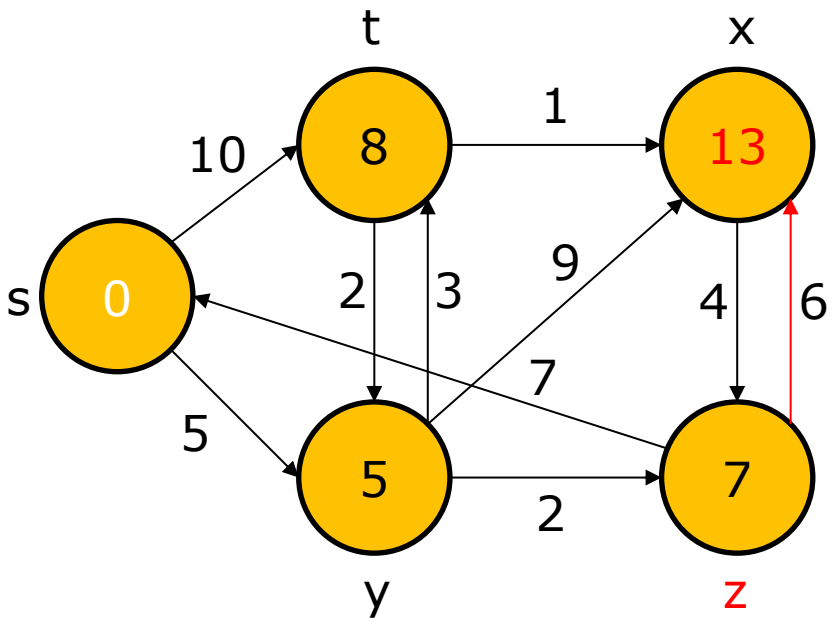



S = {s, y, z}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

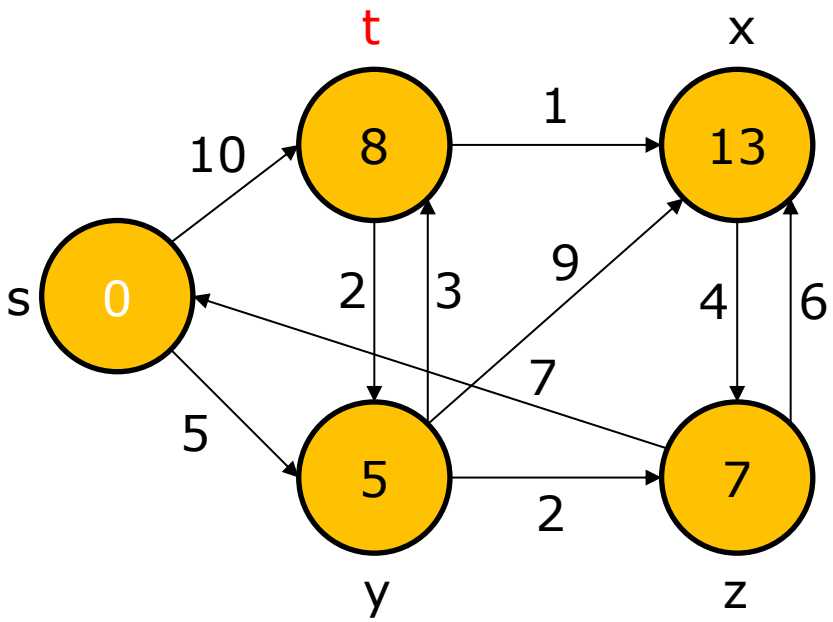



S = {s, y, z}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | 0 | 8 | 13 | 5 | 7 |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | Nil | y | z | s | y |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

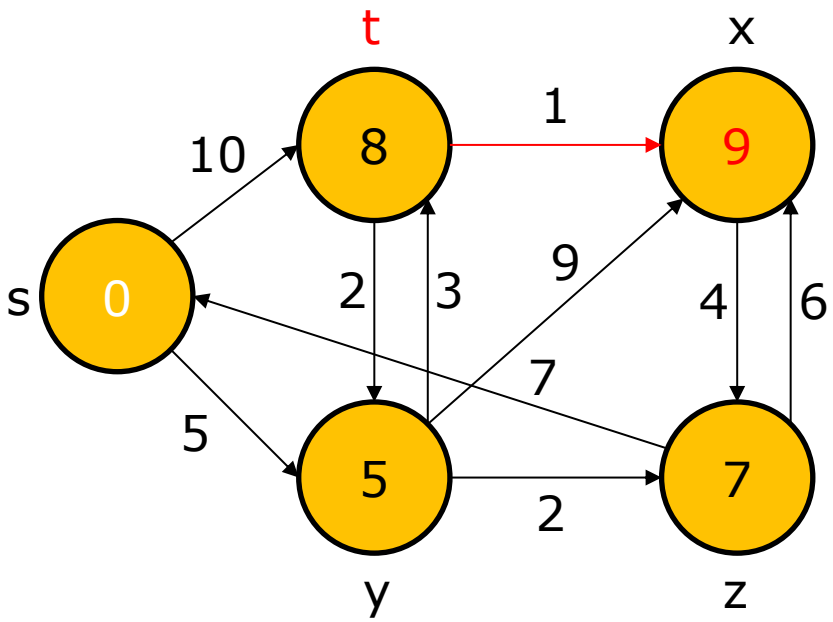



S = {s, y, z, t}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | 0 | 8 | 13 | 5 | 7 |
| 4 | | | | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | Nil | y | z | s | y |
| 4 | | | | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

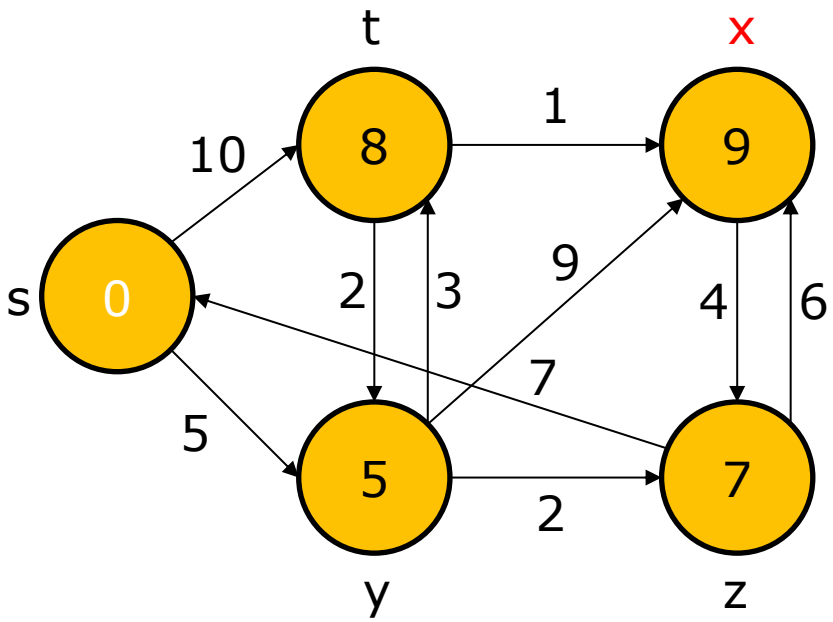



S = {s, y, z, t}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | 0 | 8 | 13 | 5 | 7 |
| 4 | | | 9 | | |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | Nil | y | z | s | y |
| 4 | | | t | | |
| 5 | | | | | |

# Dijkstra's algorithm: Example

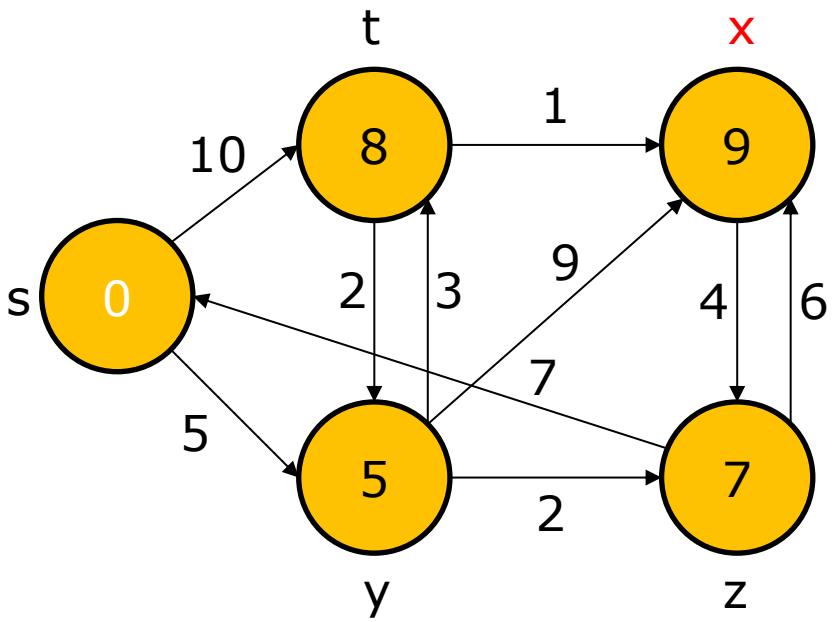



S = {s, y, z, t, x}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | 0 | 8 | 13 | 5 | 7 |
| 4 | 0 | 8 | 9 | 5 | 7 |
| 5 | | | | | |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | Nil | y | z | s | y |
| 4 | Nil | y | t | s | y |
| 5 | | | | | |

# Dijkstra's algorithm: Example




S = {s, y, z, t, x}

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | 0 | 10 | ∞ | 5 | ∞ |
| 2 | 0 | 8 | 14 | 5 | 7 |
| 3 | 0 | 8 | 13 | 5 | 7 |
| 4 | 0 | 8 | 9 | 5 | 7 |
| 5 | 0 | 8 | 9 | 5 | 7 |

| | s | t | x | y | z |
|---|---|---|---|---|---|
| 1 | Nil | s | Nil | s | Nil |
| 2 | Nil | y | y | s | y |
| 3 | Nil | y | z | s | y |
| 4 | Nil | y | t | s | y |
| 5 | Nil | y | t | s | y |

## Reading

- Sections 22.1 and 22.3.